

Common Instructions: For this assignment you will need to write a program to solve a problem, based on handout code provided on D2L. You should upload your solution to D2L in individual plaintext files with the correct extension for source code (i.e. `.py` for Python, `.c` or `.h` for C). In general, you do not need to submit handout code files that are unchanged; you only need to upload code files that you modified in some way.

Project Description: In this problem you will complete the Python program `bridge.py`, which is available on D2L. It simulates a one-lane bridge, on which cars can only cross in one direction at a time. Cars are represented by threads, and the bridge is represented by the `OneLaneBridge` object. A `Car` thread will call the `cross` method of `OneLaneBridge` before attempting to cross the bridge, supplying a “direction” argument indicating which direction it wants to cross in (north or south). When a car finishes crossing the bridge, it calls the `finished` method.

You must finish implementing the `OneLaneBridge` object so that these two methods guarantee that cars are only crossing the bridge in one direction at a time. In other words, if there are currently cars crossing the bridge in the northbound direction, then any `Car` thread that calls `cross` with an argument of `SOUTH` should block until the bridge is clear, but any `Car` thread that calls `cross` with an argument of `NORTH` should be allowed to proceed without blocking. If the bridge is currently empty (has no cars on it), then a `Car` thread that attempts to cross in either direction should succeed.

Part 1: Implement the `OneLaneBridge` methods using semaphores. Remember that in Python, a semaphore’s **P** operation is named `acquire`, and the **V** operation is named `release`. The methods you need to implement are marked with the comment `# TODO`. You only need to submit your code for the `OneLaneBridge` object, but you may submit the entire (completed) `bridge.py` file if you want.

Part 2: Implement the `OneLaneBridge` methods using monitor locks and condition variables (in other words, turn the `OneLaneBridge` object into a monitor object). You only need to submit your code for the `OneLaneBridge` object, but you may submit the entire (completed) `bridge.py` file if you want.